UNIT 3

1) Activity Diagram for Passport Management System

What is Activity Diagram?

- It shows **step-by-step workflow** of a system.
- It explains how activities move from **start to end**.
- Swim lanes divide work between different people/departments.

Swim Lanes Used

- Applicant
- Passport Office
- Police Department

Flow of Process

1. Applicant applies online.
2. Gets appointment date.
3. Submits documents.
4. Passport office checks documents.
5. If documents are wrong → Reapply.
6. If correct → Police verification.
7. If police verification fails → Reapply.
8. If successful → Passport issued.

Important Point

- Decision nodes check:
    - Documents valid?
    - Verification successful?

Easy Memory Trick

Apply → Appointment → Document Check → Police Verification → Passport Issue

---

2) Difference Between Sequence Diagram & Collaboration Diagram

Basis	Sequence Diagram	Collaboration Diagram
Focus	Time sequence	Object relationship
Structure	Vertical	Network style
Shows	What happens first/next	Which objects communicate
Time Representation	Top to bottom	Message numbering
Main Elements	Lifelines, messages	Objects, links

Easy Understanding

- **Sequence Diagram** = "When messages happen"
- **Collaboration Diagram** = "Who communicates with whom"

Example

Online shopping:

- Customer places order
- System processes payment
- Confirmation sent

---

3) State Machine Diagram for Washing Machine

What is State Diagram?

- Shows different **states** of a system.
- Shows movement from one state to another.

Main States

1. Idle
2. Filling Water
3. Washing
4. Rinsing
5. Spinning
6. End

Special Condition

- Door open/error → Pause state

Flow

Idle → Fill Water → Wash → Rinse → Spin → End

Easy Memory Trick

Fill → Wash → Rinse → Spin

---

4) Sequence Diagram for Meeting Scheduler

Main Objects

- Project Leader
- GUI Form
- Scheduler
- Database
- Mobile Gateway
- Members

Working

1. Leader enters meeting details.
2. Scheduler checks free slots.
3. Database returns available time.
4. Scheduler confirms meeting.
5. SMS sent through mobile gateway.
6. Members receive invitation.

Easy Flow

Enter Details → Check Slots → Confirm → Send SMS

---

5) State Diagram Concepts

---

i) Entry and Exit Actions

Meaning

- Entry action runs when entering a state.
- Exit action runs when leaving a state.

Example

Washing Machine:

- Entry → Start motor
- Exit → Stop motor

Syntax

- entry / action
- exit / action

---

ii) History States

Meaning

- Remembers previous state.
- Represented by **H**.

Example

Music player resumes last song after pause.

---

iii) Nested States

Meaning

- States inside another state.

Example

Washing Process:

- Wash
- Rinse
- Spin

---

iv) Activities

Meaning

- Continuous work inside a state.
- Written as:
    - do / activity

Example

Downloading file continuously.

---

6) Message Notations in Sequence Diagram

i) Synchronous Message

- Sender waits for reply.
- Filled arrow.

Example:
ATM validates PIN.

---

ii) Asynchronous Message

- Sender does not wait.
- Open arrow.

Example:
Sending SMS.

---

iii) Return Message

- Shows response.
- Dashed arrow.

Example:
Payment success message.

---

iv) Create Object Message

- Creates new object.

Example:
Create new account.

---

v) Destroy Object Message

- Ends object life.
- Marked with X.

Example:
Delete session after logout.

---

7) State Diagram for Coffee Vending Machine

Main States

1. Idle
2. Accept Money
3. Check Ingredients
4. Prepare Coffee
5. Dispense Coffee
6. End

Conditions

- Insufficient money → Return money
- No ingredients → Out of stock

Flow

Select Coffee → Insert Money → Check Ingredients → Prepare → Dispense

Easy Memory Trick

Select → Pay → Prepare → Serve

---

8) UML Diagrams Used for Dynamic Modelling

Diagram	**Purpose**
Activity Diagram	Flow of process
Sequence Diagram	Time-based interaction

Diagram	Purpose
Collaboration Diagram	Object communication
State Diagram	State changes
Timing Diagram	Time constraints

Easy Remember

- Activity → Workflow
- Sequence → Message order
- State → State changes

---

9) Swim Lanes

Meaning

- Partitions in activity diagram.
- Show responsibility of each actor.

Uses

1. Shows who performs task.
2. Improves readability.
3. Helps understand workflow.

Example

Online Shopping:

- Customer places order
- System processes order
- Payment gateway handles payment

Easy Definition

Swim lanes divide activities according to responsibility.

---

10) Sequence Diagram for ATM Invalid PIN

Main Objects

- Customer
- ATM
- Bank Server

Flow

1. Insert card
2. Enter PIN
3. ATM sends PIN for checking
4. Bank server says invalid PIN
5. ATM shows error
6. After 3 wrong attempts → Card blocked

Easy Memory Trick

Insert → Enter PIN → Validate → Error → Block Card

---

11) Activity Diagram for Exam Form Registration

Swim Lanes

- Student
- Exam Portal
- Accounts Department

Main Flow

1. Login
2. Fill form
3. Eligibility check
4. Fork:
    - Fee payment
    - Form verification
5. Join
6. Registration confirmed

Constraints

- Attendance required
- No pending fees

Fork-Join Meaning

- Fork → Parallel tasks start
- Join → Parallel tasks complete together

---

12) Message Notations (Repeated Question)

Quick Revision

Message Type	**Meaning**
Synchronous	Wait for reply
Asynchronous	No waiting
Return	Response message
Create	New object created
Destroy	Object terminated

---

13) Sequence Diagram for Online Purchase

Main Objects

- Customer
- Website
- Inventory
- Payment Gateway
- Order System

Flow

1. Browse product
2. Add to cart
3. Checkout
4. Check stock

5. Payment processing
6. Create order
7. Send confirmation

Easy Memory Trick

Browse → Cart → Pay → Confirm

---

14) Advanced State Diagram Concepts

---

i) Concurrent States

Meaning

- Multiple states active together.

Example

Mobile downloading file while playing music.

---

ii) Nested States

Meaning

- States inside another state.

Example

Wash → Rinse → Spin

---

iii) Composite States

Meaning

- Parent state containing many sub-states.

Example

Order Processing:

- Validation
- Payment
- Shipping

---

iv) Sub-machine States

Meaning

- Reuse another state machine.

Example

ATM transaction process reused.

Very Important Exam Keywords

Topic	**Keywords**
Activity Diagram	Workflow, decision, swim lane
Sequence Diagram	Lifeline, message flow
State Diagram	State, transition
Swim Lane	Responsibility
Fork-Join	Parallel processing
Synchronous	Wait
Asynchronous	No wait
History State	Remember previous state

UNIT 4

1) Object Relational Mapping (ORM) and its Capabilities

What is ORM?

- ORM converts **objects into database tables**.
- It connects **object-oriented programming** with **relational databases**.
- Developers can work with objects instead of SQL queries.

Basic Mapping

- Class → Table
- Object → Row
- Attribute → Column

Example

- Student class → Student table
- Student object → One record in table

Capabilities of ORM

1. Persistence

- Saves objects into database.
- Supports CRUD operations.

2. Retrieval

- Fetches data without writing SQL.

3. Relationship Handling

- Manages one-to-one and one-to-many relationships.

4. Transaction Management

- Supports commit and rollback.

5. Lazy/Eager Loading

- Lazy → Load when needed.
- Eager → Load immediately.

6. Caching

- Stores frequently used data temporarily.

7. Validation

- Checks data rules before saving.

8. Schema Generation

- Automatically creates database tables.

Easy Memory Trick

Map → Save → Retrieve → Manage Relations

---

2) Activities in Access Layer Design Process

What is Access Layer?

- Layer between business logic and database.
- Handles all database operations.

Main Activities

1. Identify Data Sources

- Find database/files/external systems.

2. Define Data Requirements

- Decide what data is needed.

3. Design Access Classes

- Create DAO classes.

4. Implement ORM

- Map objects with tables.

5. Manage Connections

- Handle database connections efficiently.

6. Transaction Handling

- Maintain consistency using commit/rollback.

7. Error Handling

- Handle database failures.

8. Security

- Prevent SQL injection and unauthorized access.

9. Performance Optimization

- Use caching and indexing.

10. Testing

- Verify all database operations.

Easy Flow

Identify → Design → Connect → Secure → Test

---

3) Steps in View Layer Macro Process

What is View Layer?

- Presentation layer for user interaction.
- Shows forms, pages, screens.

Steps

1. Identify User Requirements

- Understand user needs.

2. Define UI Structure

- Design pages and layout.

3. Design Components

- Create buttons, forms, tables.

4. Map Data to Views

- Display backend data on UI.

5. Define Navigation

- Design movement between pages.

6. Handle User Input

- Accept and validate inputs.

7. Apply UI Principles

- Maintain consistency and simplicity.

8. Integrate Business Logic

- Connect UI with backend.

9. Test UI

- Check usability and performance.

Easy Memory Trick

Requirements → UI → Navigation → Validation → Testing

---

4) Main Activities in Design Process

What is Design Process?

- Converts requirements into system structure.

Activities

1. Architectural Design

- Defines system structure.

2. Data Design

- Designs database and relationships.

3. Interface Design

- Designs UI and APIs.

4. Component Design

- Breaks system into modules.

5. Deployment Design

- Defines hardware setup.

6. Database Design

- Creates schema and tables.

7. Security Design

- Adds authentication and protection.

8. Error Handling

- Handles failures properly.

9. Performance Optimization

- Improves speed and efficiency.

10. Validation & Review

- Checks correctness of design.

Easy Memory Trick

Architecture → Data → Interface → Components → Security

---

5) OCL and Expressions

What is OCL?

- Object Constraint Language.
- Used to define rules and constraints in UML.

What are Expressions?

- Statements that return values or conditions.

Common OCL Expressions

1. Invariant

- Condition always true.

Syntax:

```
context Student
inv: age > 0
```

---

2. Precondition

- Condition before operation.

Syntax:

```
pre: amount > 0
```

---

3. Postcondition

- Condition after operation.

Syntax:

post: balance = balance@pre - amount

---

4. Select

- Filters collection.

students->select(age > 18)

---

5. Size

students->size()

---

6. ForAll

students->forAll(age > 18)

---

7. Exists

students->exists(age < 18)

Easy Memory Trick

Invariant → Pre → Post → Select

---

6) Design Process of Access Layer Classes

Assumed Example

Online Shopping System

DAO Classes

- ProductDAO
- CustomerDAO
- OrderDAO

Steps

1. Identify Entities

- Product, Customer, Order.

2. Create DAO Classes

- Separate class for each entity.

3. Define CRUD

- Insert, Update, Delete, Select.

4. Establish DB Connection

- Connect with database.

5. Apply ORM

- Map classes with tables.

6. Handle Transactions

- Commit and rollback.

7. Error Handling

- Manage query failures.

8. Optimize Performance

- Use caching/indexing.

9. Testing

- Validate operations.

Easy Flow

Entity → DAO → CRUD → Connect → Test

---

7) Deployment Diagram for Railway Reservation System

What is Deployment Diagram?

- Shows physical arrangement of hardware and software.

Main Nodes

- Client Device

- Web Server
- Application Server
- Database Server
- Payment Gateway

Working

1. Client

- User searches/book tickets.

2. Web Server

- Handles UI requests.

3. Application Server

- Processes booking logic.

4. Database Server

- Stores train and ticket data.

5. Payment Gateway

- Handles online payments.

Easy Flow

Client → Web Server → App Server → Database

---

8) OCL Pre and Post Conditions

Precondition

- Must be true before operation.

Syntax

context Account::withdraw(amount)
pre: amount > 0

Postcondition

- Must be true after operation.

Syntax

context Account::withdraw(amount)
post: balance = balance@pre - amount

Example

ATM Withdrawal:

- Pre → Sufficient balance.
- Post → Updated balance.

Easy Memory Trick

Pre = Before, Post = After

---

9) Macro-Level Process of Identifying View Layer Classes

Meaning

- Identifies UI classes using use cases.

Steps

1. Analyze Use Cases

- Understand user actions.

2. Identify UI Boundaries

- Find interaction points.

3. Identify View Classes

Examples:

- LoginPage
- CartPage
- PaymentPage

4. Assign Responsibilities

- Display and validation.

5. Map Use Case Steps

- Convert actions into screens.

6. Define Navigation

- Movement between pages.

7. Refine Design

- Improve usability.

Easy Memory Trick

Use Case → UI → Classes → Navigation

---

10) Steps in Designing Business Classes

What are Business Classes?

- Represent business logic and entities.

Examples:

- Customer
- Account
- Order

Steps

1. Identify Entities

- Find domain objects.

2. Define Attributes

- Define data members.

3. Define Methods

- Define operations/functions.

4. Establish Relationships

- Association, inheritance.

5. Apply Business Rules

- Add constraints.

6. Design Interfaces

- Public methods.

7. Refine Design

- Remove redundancy.

8. Validate with Use Cases

- Ensure all requirements covered.

9. Optimize Reusability

- Make reusable classes.

Easy Memory Trick

Entity → Attribute → Method → Relationship

---

11) Use of Activity Diagram in Business Class Design

Importance

1. Understand Workflow

- Shows sequence of activities.

2. Identify Methods

- Activities become methods.

3. Identify Classes

- Entities in workflow become classes.

4. Shows Decisions

- Business rules become conditions.

5. Supports Parallel Processing

- Fork/join for simultaneous tasks.

6. Improves Communication

- Easy visual understanding.

7. Covers Use Cases

- Ensures complete functionality.

Example

Order Processing:
Select → Pay → Confirm

Easy Memory Trick

Workflow → Methods → Classes

---

12) Benefits and Tasks of Access Layer Class

Benefits

1. Separation of Concerns

- Separates database logic.

2. Maintainability

- Easy updates.

3. Reusability

- Reuse database code.

4. Data Abstraction

- Hides SQL complexity.

5. Security

- Prevents direct DB access.

6. Better Performance

- Uses caching.

7. Easier Testing

- Independent testing possible.

Tasks

1. Connection Management

2. Data Retrieval

3. Data Insertion

4. Data Update

5. Data Deletion

6. Transaction Management

7. Error Handling

8. ORM Mapping

9. Security Handling

10. Performance Optimization

Easy Memory Trick

Connect → Retrieve → Insert → Update → Delete

---

13) OCL Mechanisms and Applications

Mechanisms

1. Invariants

- Always true conditions.

2. Preconditions

- Before operation.

3. Postconditions

- After operation.

4. Derivation Rules

- Compute derived values.

5. Query Expressions

- Retrieve/filter data.

6. Association Constraints

- Rules on relationships.

Applications

1. Model Validation

2. Business Rule Definition

3. Database Constraints

4. Documentation

5. Test Case Generation

6. Code Generation

Easy Memory Trick

Validate → Rule → Constraint → Test

---

14) Steps in Designing View Layer

Steps

1. Analyze User Requirements

2. Identify View Classes

3. Design UI Structure

4. Design Components

5. Map Data to UI

6. Define Navigation

7. Handle Input & Validation

8. Apply UI Principles

9. Integrate Business Logic

10. Test & Refine

Example

Login → Product → Cart → Checkout

Easy Memory Trick

Requirement → UI → Navigation → Validation → Testing

---

15) Deployment Diagram for Hotel Management System

Main Nodes

- Client Devices
- Web Server
- Application Server
- Database Server

- Payment Gateway

Working

1. Client Devices

- Receptionist/Admin/Customer uses system.

2. Web Server

- Handles requests and pages.

3. Application Server

- Booking, billing, reports.

4. Database Server

- Stores customer and room data.

5. Payment Gateway

- Processes online payments.

Easy Flow

Client → Web Server → App Server → Database → Payment

Important Exam Keywords

Topic	**Keywords**
ORM	Mapping objects to tables
Access Layer	Database interaction
View Layer	User interface
OCL	Constraints and rules
Business Class	Core logic
Deployment Diagram	Physical architecture
DAO	Data Access Object

UNIT 5

1) Factory, State & Adapter Design Patterns

a) Factory Design Pattern

Definition

- Factory pattern creates objects without exposing exact class details.
- Object creation is done through a factory method.

Easy Points

1. Hides object creation logic.
2. Client uses factory instead of new.
3. Creates objects based on condition/input.
4. Provides flexibility and loose coupling.

Example

- **Payment System**
  - Credit Card
  - UPI
  - Net Banking
- Factory creates required payment object.

Remember

👉 **Factory = Object Creation**

---

b) State Design Pattern

Definition

- State pattern changes object behavior according to its current state.

Easy Points

1. Different states have different behaviors.
2. Each state is represented by a separate class.
3. Avoids large if-else conditions.

4. Object changes behavior dynamically.

Example

- **ATM Machine**
    - Idle
    - Card Inserted
    - Transaction
    - Out of Service

Remember

👉 **State = Behavior changes with state**

---

c) Adapter Design Pattern

Definition

- Adapter pattern connects incompatible interfaces.

Easy Points

1. Works as a bridge between systems.
2. Converts one interface into another.
3. Reuses old code without modification.
4. Useful in third-party API integration.

Example

- Mobile charger adapter
- Payment gateway integration

Remember

👉 **Adapter = Compatibility**

---

2) GoF (Gang of Four) Patterns

Definition

- 23 standard design patterns introduced by four authors.

Categories

Creational

- Singleton
- Factory
- Builder

Structural

- Adapter
- Facade
- Proxy

Behavioral

- Strategy
- State
- Observer

---

a) Singleton Pattern

Definition

- Only one object of class exists.

Points

1. Single instance.
2. Global access point.
3. Saves resources.

Example

- Database connection manager

Remember

👉 **Singleton = One object only**

---

b) Strategy Pattern

Definition

- Multiple algorithms can be selected at runtime.

Points

1. Algorithms stored separately.
2. Common interface used.
3. Flexible and reusable.

Example

- Different payment methods

Remember

👉 **Strategy = Change algorithm**

---

c) Adapter Pattern

Definition

- Converts incompatible interfaces.

Example

- Media player supporting MP3/MP4/VLC

Remember

👉 **Adapter = Interface converter**

---

3) GRASP Patterns

Definition

- Guidelines for assigning responsibilities in OOP.

Important Patterns

1. Information Expert

- Responsibility given to class having required data.

👉 Example: Order calculates total.

---

2. Creator

- Class creates objects it uses.

👉 Example: Order creates OrderItem.

---

3. Controller

- Handles system requests.

👉 Example: OrderController

---

4. Low Coupling

- Reduce dependency between classes.

---

5. High Cohesion

- One class should do related work only.

---

6. Polymorphism

- Same method behaves differently.

👉 Example: Different pay() methods.

---

7. Pure Fabrication

- Artificial helper class created.

👉 Example: DatabaseHelper

---

8. Indirection

- Intermediate class reduces dependency.

---

9. Protected Variations

- Protect system from future changes using interfaces.

---

4) Strategy vs State Pattern

Strategy Pattern

Definition

- Select algorithm dynamically.

Example

- Payment methods

Remember

👉 **Strategy = Different ways to do task**

State Pattern

Definition

- Behavior changes according to state.

Example

- ATM machine states

Remember

👉 **State = Different states of object**

Difference Table

Feature	**Strategy**	**State**
Purpose	Change algorithm	Change behavior
Controlled by	Client	State transition
Example	Payment methods	ATM states

---

5) Information Expert Pattern

Definition

- Responsibility assigned to class having required data.

Points

1. Keeps data and behavior together.
2. Improves encapsulation.
3. Reduces coupling.

4. Improves cohesion.

Example

- Student calculates average marks.

Remember

👉 **Expert = Class with information**

---

6) Pure Fabrication & Protected Variations

a) Pure Fabrication

Definition

- Artificial class created for better design.

Example

- DatabaseHelper

Benefits

1. Low coupling
2. High cohesion
3. Better reusability

Remember

👉 **Fabrication = Helper class**

---

b) Protected Variations

Definition

- Protect system from changes using interfaces.

Example

- Payment interface for UPI/Card.

Benefits

1. Easy modification
2. Flexible system
3. Less impact of changes

Remember

👉 **Protected Variations = Future safety**

---

7) Usage of Singleton, Adapter, Facade

i) Singleton

Usage

- Database manager
- Logger
- Cache system

👉 One shared object.

---

ii) Adapter

Usage

- Third-party API integration
- Legacy system connection

👉 Makes incompatible systems work together.

---

iii) Facade

Usage

- Simplifies complex subsystem.

Example

- Home theater system

👉 One simple interface for many operations.

---

8) Behavioral Pattern & Iterator Pattern

Behavioral Pattern

Definition

- Focuses on object communication and behavior.

Examples

- Strategy
- State
- Iterator

Iterator Pattern

Definition

- Access collection elements sequentially without exposing structure.

Points

1. Uniform traversal
2. Hides internal structure
3. Works for array/list/tree
4. Reduces coupling

Example

- Music playlist traversal

Remember

👉 **Iterator = Sequential traversal**

---

9) Facade Pattern Solution

Definition

- Provides simple interface to complex subsystem.

Solution

1. Create Facade class.
2. Client talks only to Facade.
3. Facade handles subsystem internally.

Example

- Home theater watchMovie() method.

Remember

👉 **Facade = Simplified interface**

---

10) Structural Design Pattern Solution

Definition

- Organizes classes and objects efficiently.

Main Solutions

1. Use composition
2. Reduce coupling
3. Reuse existing classes
4. Simplify structure

Examples

- Adapter
- Facade
- Decorator
- Proxy

Remember

👉 **Structural = Class organization**

---

11) Iterator Pattern with UML

Definition

- Sequentially accesses collection elements.

Uses

1. Common traversal method
2. Hides internal structure
3. Multiple traversals possible

UML Components

1. Iterator
2. Concrete Iterator

3. Aggregate
4. Client

Example

- Playlist system

Remember

👉 **Iterator = Common traversal method**

---

12) Design Pattern & Categories

Definition

- Reusable solution to common software problems.

Categories

1. Creational Pattern

- Object creation related.

👉 Example: Factory

---

2. Structural Pattern

- Class/object arrangement.

👉 Example: Adapter

---

3. Behavioral Pattern

- Object interaction and behavior.

👉 Example: Strategy

Final Quick Revision

Pattern	**Main Idea**
Factory	Object creation

Pattern	Main Idea
State	State-based behavior
Adapter	Compatibility
Singleton	Single object
Strategy	Change algorithm
Facade	Simplify system
Iterator	Collection traversal
GRASP	Responsibility assignment

UNIT 6

1) Need for Software Architectural Styles

What is Software Architectural Style?

It is a standard way of organizing software components and their communication.

Need for Architectural Styles

1. Standard Structure

- Gives a proper structure for software development.
- Makes system design organized and consistent.

2. Reusability

- Reuses already proven designs.
- Saves development time and cost.

3. Easy Communication

- Developers and designers can understand system easily.
- Works like a common language.

4. Maintainability

- Easy to update and debug system.
- Changes in one module affect less on others.

5. Quality Improvement

- Helps achieve security, performance, scalability, reliability.

6. Reduces Complexity

- Breaks large system into smaller parts.
- Easier to manage and understand.

Common Architectural Styles

Style	Meaning	Example
Layered	System divided into layers	Web applications

Style	Meaning	Example
Client-Server	Client requests, server responds	Online banking
Object-Oriented	System made of objects	Java applications
Component-Based	Uses reusable components	E-commerce system
SOA	Uses independent services	Cloud apps
Event-Driven	Responds to events	GUI applications
Pipe & Filter	Data flows through filters	Compiler

---

2) SOAP Messaging and Atomic Transaction

SOAP

SOAP is an XML-based protocol used for communication between web services.

SOAP Messaging Process

1. Message Creation

- Client creates SOAP message in XML.
- Contains Envelope, Header, Body.

2. Envelope Wrapping

- Message is wrapped inside SOAP envelope.
- Defines message boundaries.

3. Transmission

- Message sent through HTTP/HTTPS.

4. Server Processing

- Server reads SOAP message.
- Processes request.

5. Response

- Server sends response back to client.

SOAP Structure

- Envelope → Main container

- Header → Extra information
- Body → Actual data

Atomic Transaction

Atomic transaction means:
👉 Either all operations succeed or all fail.

Process

1. Start Transaction

- Coordinator starts transaction.

2. Execute Operations

- Multiple services perform tasks.

3. Prepare Phase

- Services check if they can complete successfully.

4. Commit/Rollback

- All YES → Commit
- Any NO → Rollback

5. Completion

- Transaction fully completed or cancelled.

Example

Online booking:

- Flight booking
- Payment
- Seat reservation

If payment fails → whole booking cancelled.

---

3) Component-Based Software Architecture (CBSA)

Definition

CBSA builds software using reusable independent components.

Main Concepts

1. Component

- Self-contained software unit.
- Performs specific task.

2. Interface

- Helps components communicate.

3. Reusability

- Same component reused in many systems.

4. Replaceability

- One component can be replaced easily.

Characteristics

- Modularity
- Loose coupling
- Reusability
- Interoperability

Advantages

- Faster development
- Easy maintenance
- Scalable
- Flexible

Disadvantages

- Integration difficulty
- Debugging complexity
- Performance overhead

Example

E-commerce:

- Payment module
- Cart module

- User module

---

4) Software Architecture and Design Levels

Software Architecture

Definition

High-level structure of software system.

Functions

- Defines components
- Defines communication
- Focuses on quality attributes

Software Design Levels

1. Architectural Design

- Overall system structure.
- Example: Client-server design.

2. High-Level Design (HLD)

- Divides system into modules.
- Example: Login, Transaction module.

3. Low-Level Design (LLD)

- Internal logic of modules/classes.
- Example: deposit(), withdraw() methods.

Hierarchy

Architecture → HLD → LLD → Coding

---

5) i) Data Abstraction & Object-Oriented Organization

Data Abstraction

- Hides internal details.
- Shows only important features.

Object-Oriented Organization

- System built using objects.

Features

1. Encapsulation

- Data + methods together.

2. Inheritance

- Reuse properties of existing class.

3. Message Passing

- Objects communicate using methods.

Example

Banking System:

- Customer object
- Account object

---

ii) Pipes and Filters Architecture

Definition

System divided into filters connected by pipes.

Features

1. Filters

- Process data.

2. Pipes

- Transfer data.

3. Sequential Flow

- Output of one filter becomes input of next.

4. Independent Filters

- One filter change does not affect others.

Example

Compiler:
Lexical → Syntax → Semantic → Code Generation

---

6) Short Notes

a) Software Product Line Architecture (SPLA)

Definition

Develops multiple related products using common architecture.

Features

- Core reusable assets
- Product customization
- High reuse
- Domain-based design

Example

Microsoft Office Suite

---

b) Real-Time Software Architecture

Definition

System where response time is very important.

Features

- Time constraints
- Predictable behavior
- Priority scheduling
- Event-driven processing

Example

- Air traffic control
- ABS braking system

---

c) Component-Based Software Architecture

Features

- Independent components
- Reusable modules
- Loose coupling
- Easy maintenance

Example

E-commerce modules

---

7) Client/Server Software Architecture

Definition

Client requests services, server provides services.

Components

1. Client

- User interface
- Sends requests

2. Server

- Processes data
- Handles database

3. Communication Layer

- Uses HTTP/TCP-IP

Types

1. 2-Tier

- Client directly connected to server.

2. 3-Tier

- Client + Application Server + Database

3. N-Tier

- Multiple layers.

Working

1. Client sends request
2. Server processes
3. Response returned

Advantages

- Centralized control
- Security
- Scalability

Disadvantages

- Server overload
- Network dependency

---

8) Object-Oriented Software Architecture (OOSA)

Definition

System organized using interacting objects.

Key Features

1. Abstraction

- Hide implementation details.

2. Encapsulation

- Protect data.

3. Inheritance

- Reuse existing class features.

4. Polymorphism

- Same function behaves differently.

5. Modularity

- Divides system into modules.

6. Reusability

- Reuse classes and objects.

7. Message Passing

- Objects communicate.

8. Dynamic Binding

- Method selected at runtime.

Example

Banking system objects:

- Customer
- Account
- Transaction

---

9) Entities in Documenting Architectural Pattern

1. Pattern Name

- Name of pattern.

2. Intent

- Purpose of pattern.

3. Context

- Situation where used.

4. Problem

- Problem solved by pattern.

5. Solution

- How pattern solves problem.

6. Structure

- Arrangement of components.

7. Participants

- Components involved.

8. Collaboration

- Interaction between components.

9. Consequences

- Advantages and disadvantages.

10. Implementation

- Guidelines for implementation.

11. Known Uses

- Real-world examples.

12. Related Patterns

- Similar patterns.

---

10) Service-Oriented Architecture (SOA)

Definition

System built using independent services communicating over network.

Features

1. Independent Services

- Each service performs one task.

2. Loose Coupling

- Services work independently.

3. Standard Communication

- Uses SOAP, REST, HTTP.

4. Reusability

- Service reused in many applications.

5. Interoperability

- Different technologies can communicate.

Applications

1. E-Commerce

- Payment service
- Order service

2. Banking

- Fund transfer
- Loan service

3. Healthcare

- Patient management

4. Travel Booking

- Hotel and flight booking

5. Cloud Computing

- AWS services

Advantages

- Flexible
- Scalable
- Reusable

Disadvantages

- Complex setup
- Performance overhead

---

11) Multiple Views of Software Architecture

1. Logical View

- Functional structure.
- Uses classes and objects.

2. Process View

- Runtime behavior.
- Processes and threads.

3. Development View

- Code organization.
- Modules and packages.

4. Physical View

- Hardware deployment.

5. Scenario View

- User interaction/use cases.

4+1 Model

Logical + Process + Development + Physical + Scenario

---

12) Product Line Software Architecture (PLSA)

Definition

Developing related products using common architecture and reusable assets.

Features

1. Core Assets

- Shared architecture and code.

2. Variability

- Different features for different products.

3. Reusability

- Reuse components.

4. Domain-Specific

- Same application domain.

5. Managed Evolution

- Updates benefit all products.

Example

- Microsoft Office
- Mobile OS versions

Advantages

- Faster development
- Reduced cost
- Consistency

Disadvantages

- Complex initial setup
- Difficult variability management